Generative Modeling Slides

James Evans

07/02/2025

ML Project: Generative modeling

Ricardo Baptista, Suvedei Soyolerdene, Giulio Trigila, and Tanya Wang

Caltech, Baruch College CUNY, and NYU

PolyMathJr Summer Program

June 20, 2025

Probabilistic modeling

- Given data $\{\mathbf{z}^i\}$ sampled from an unknown probability density ρ, we would like to estimate $\rho(\mathbf{z})$
- Example: what is the distribution of the heights among the high school students in NYC?
- A more complicated example: given samples $\{(\mathbf{z}^i, \mathbf{y}^i)\}$ we would like to estimate the conditional probability density $\rho(\mathbf{z}|\mathbf{y})$
- Example: given that New York is located at 40.7128° N, 74.0060° W on the sea level and that tomorrow is June 22 (these are the factors $\mathbf{y}_i \in \mathbb{R}^m$) what is the probability that the temperature (i.e., the outcome $z \in \mathbb{R}$) is going to be 25 degrees Celsius?

We use probabilistic models to describe certain outputs (e.g. from a physical system) and make predictions about the future.

Large-scale probabilistic models

- **Applications**: Numerical weather prediction, GPS tracking, infectious disease spread, financial market analysis, etc.

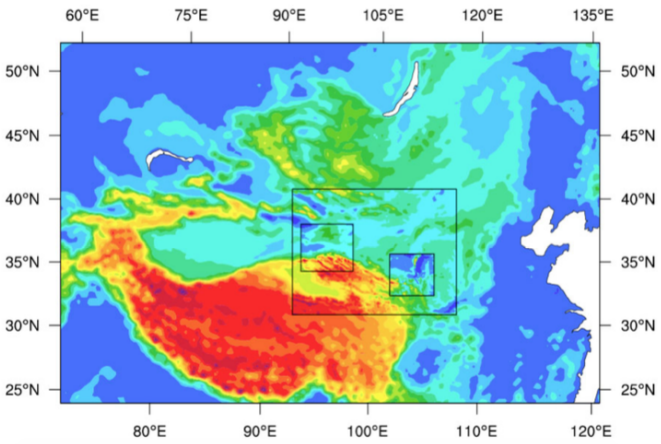


Figure: Source: NCAR ensemble wind forecast

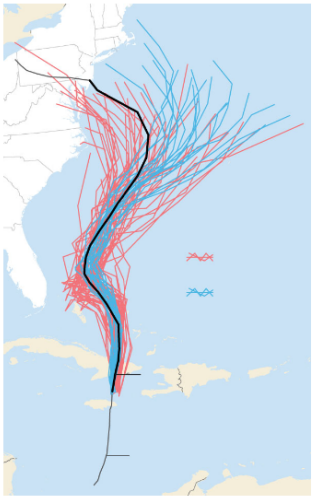


Figure: Source: Wall Street Journal

Main idea behind Mapping Methods

Most of the work related to Mapping methods is based on KL minimization

$$KL[\rho||\mu] = \int \log(\rho(x)/\mu(x))\rho(x)dx$$

- We want to find a map from ρ, known only through samples $\{x_i\}$, to a reference density $\mu = \mathcal{N}(0, I)$.
- Let $\mu = T_{\#}\tilde{\rho}$ and consider $KL[\rho||\tilde{\rho}] = KL[T] = \int \rho \log(\rho/\tilde{\rho})$
- Parametrize $T = T_\beta$, minimize $KL[T_\beta] = \text{const} - \int \rho \log(\tilde{\rho})$ over the parameters $\beta \Rightarrow \bar{\beta}$
- Use the change of variable formula to estimate ρ:

$$\rho(x) = J^G(x)\mu(G(x)) \text{ where } G = T_{\bar{\beta}}$$

*Tabak, Esteban G., and Eric Vanden-Eijnden. "Density estimation by dual ascent of the log-likelihood." Communications in Mathematical Sciences 8.1 (2010): 217-233.
**El Moselhy, Tarek A., and Youssef M. Marzouk. "Bayesian inference with optimal maps." Journal of Computational Physics 231.23 (2012): 7815-7850.

Baptista, Trigila, Wang PolyMathJr 2025: ML project 4 / 13

Detailed Explanation of Mapping Methods via KL Minimization

We are given samples $\{x_i\} \sim \rho$, where ρ is an unknown target density. The key idea of mapping methods is to estimate ρ by learning a transformation from a known reference density μ, typically $\mu = \mathcal{N}(0, I)$, using KL divergence minimization.

Starting Point: KL Divergence

The Kullback–Leibler divergence between two densities ρ and μ is defined as:

$$KL[\rho \,\|\, \mu] = \int \log\left(\frac{\rho(x)}{\mu(x)}\right) \rho(x)\, dx$$

This measures how different ρ is from μ. Since we only have access to samples from ρ, this formulation is not immediately useful.

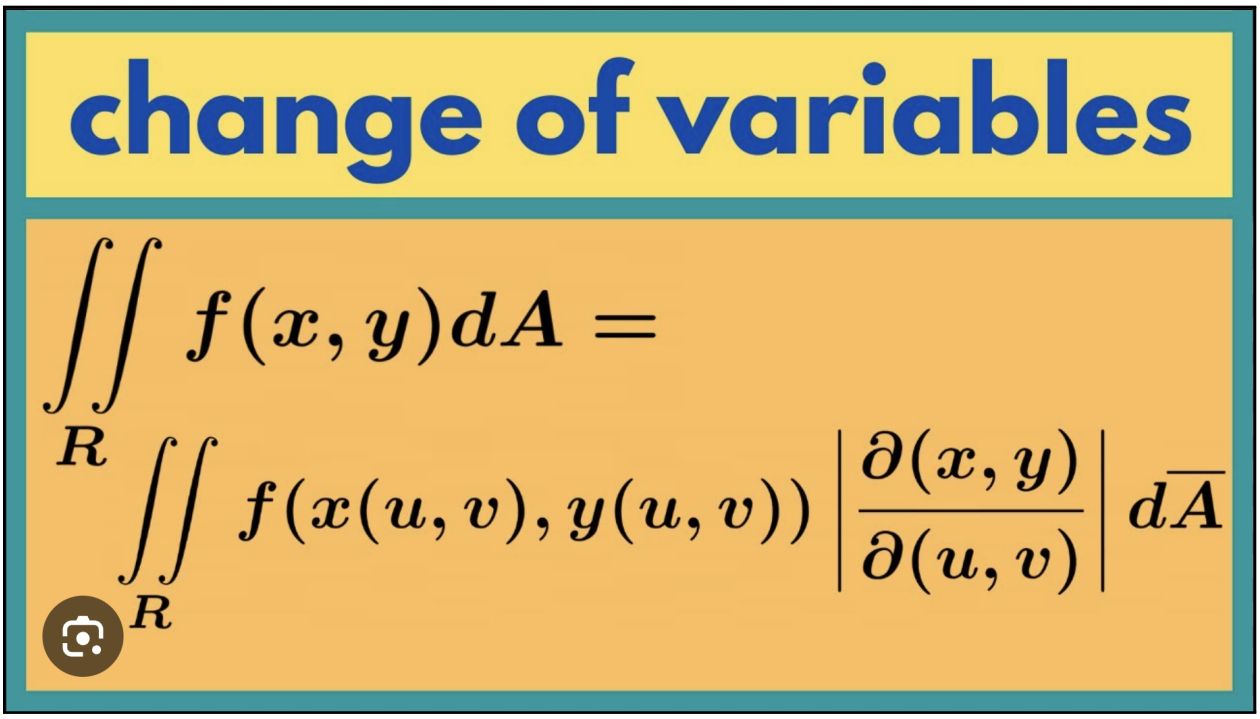
Introduce an Auxiliary Density and Pushforward Map

We define a new density $\tilde{\rho}$ on $\mathbb{R}^d$ and a smooth, invertible map $T : \mathbb{R}^d \to \mathbb{R}^d$ such that:

$$\mu = T_{\#}\tilde{\rho}$$

Here, $T_{\#}\tilde{\rho}$ denotes the *pushforward* of $\tilde{\rho}$ through T, i.e., the distribution of $T(x)$ when $x \sim \tilde{\rho}$. This construction allows us to represent the known reference density μ in terms of an unknown $\tilde{\rho}$ and a transformation T.

The change-of-variable formula tells us that:



Reformulating the KL Divergence

We now consider the Kullback–Leibler (KL) divergence between two densities:

$$\mathrm{KL}[\rho \,\|\, \mu] = \int \rho(x) \log\left(\frac{\rho(x)}{\mu(x)}\right) dx$$

Suppose $\mu = T_{\#}\tilde{\rho}$. Then, by the change of variables formula:

$$\mu(z) = \tilde{\rho}(T^{-1}(z)) \left|\det\left(\frac{\partial T^{-1}}{\partial z}(z)\right)\right|$$

To simplify notation, we now relabel variables and think of $x = z$ as the input to μ. So we define $\tilde{\rho}(x)$ as a proxy density that we want to match $\rho(x)$ against.

We now consider:

$$\mathrm{KL}[\rho \,\|\, \tilde{\rho}] = \int \rho(x) \log\left(\frac{\rho(x)}{\tilde{\rho}(x)}\right) dx$$

This KL divergence is still intractable because we don't know $\rho(x)$, only samples from it. However, we can rewrite $\tilde{\rho}(x)$ using the change-of-variable expression:

$$\tilde{\rho}(x) = \mu(T(x)) \cdot \left|\det\left(\frac{\partial T}{\partial x}(x)\right)\right|$$

This allows us to treat the KL divergence as a function of the transformation T. That is,

$$\mathrm{KL}[T] = \int \rho(x) \log\left(\frac{\rho(x)}{\mu(T(x)) \cdot \left|\det\left(\frac{\partial T}{\partial x}(x)\right)\right|}\right) dx$$

Parametric Families of Maps

In practice, we cannot optimize over all possible functions T, so we restrict ourselves to a parametric family of maps T_β, where $\beta \in \mathbb{R}^p$ is a vector of parameters (such as weights in a neural network or coefficients of a polynomial map).

Thus, we now write the divergence as:

$$KL[T_\beta] = \int \rho(x) \log \left(\frac{\rho(x)}{\mu(T_\beta(x)) \cdot \left| \det \left(\frac{\partial T_\beta}{\partial x}(x) \right) \right|} \right) dx$$

Since ρ is unknown but we have samples $\{x_i\} \sim \rho$, we approximate the expectation by empirical average:

Explaining the Matrix

The term $\frac{\partial T}{\partial x}(x)$ is the **Jacobian matrix** of the vector-valued function $T(x)$.

If:

$$T(x) = \begin{bmatrix} T_1(x) \\ T_2(x) \\ \vdots \\ T_d(x) \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

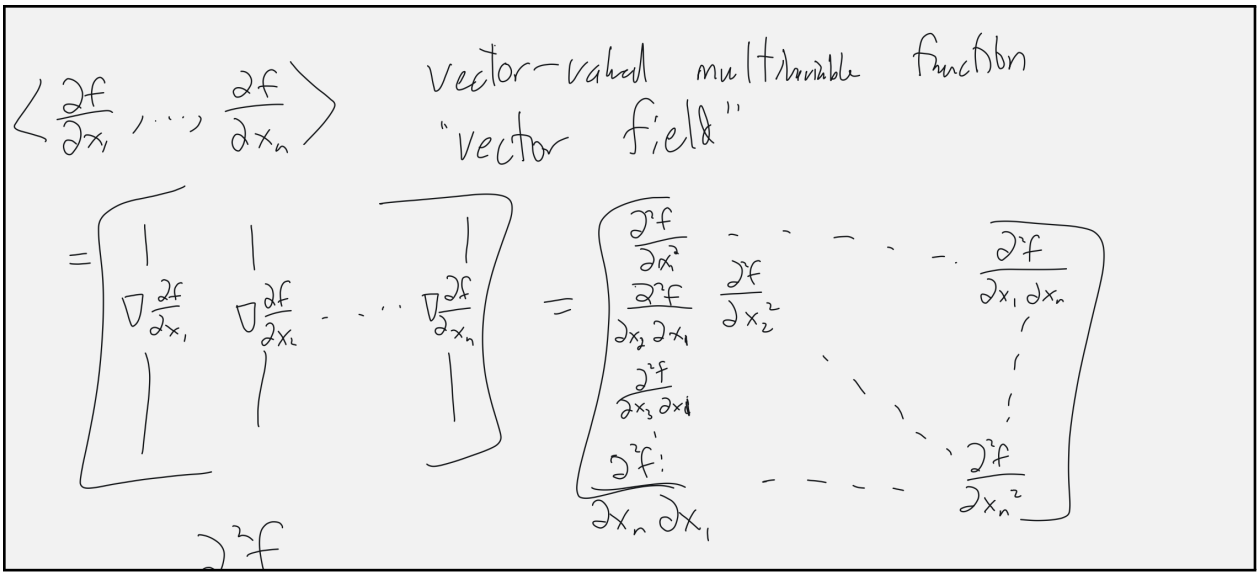
Then the Jacobian matrix is:

$$J_T(x) = \begin{bmatrix} \frac{\partial T_1}{\partial x_1} & \frac{\partial T_1}{\partial x_2} & \cdots & \frac{\partial T_1}{\partial x_d} \\ \frac{\partial T_2}{\partial x_1} & \frac{\partial T_2}{\partial x_2} & \cdots & \frac{\partial T_2}{\partial x_d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial T_d}{\partial x_1} & \frac{\partial T_d}{\partial x_2} & \cdots & \frac{\partial T_d}{\partial x_d} \end{bmatrix} \in \mathbb{R}^{d \times d}$$

Note: If $T(x) = \nabla \phi(x)$, then the Jacobian matrix of T, denoted $J_T(x)$, is exactly the Hessian matrix of the scalar function ϕ:

$$\nabla \phi(\mathbf{z}) = \begin{bmatrix} \frac{\partial \phi}{\partial z_1} \\ \frac{\partial \phi}{\partial z_2} \\ \vdots \\ \frac{\partial \phi}{\partial z_d} \end{bmatrix} \in \mathbb{R}^d$$

$$J_T(x) = D^2 \phi(x)$$



Why We Drop $\rho(x)$ in KL Divergence Approximation

We begin with the KL divergence between a true (unknown) data distribution $\rho(x)$ and a model-induced density $\tilde{\rho}(x)$:

$$KL[\rho\|\tilde{\rho}] = \int \rho(x) \log\left(\frac{\rho(x)}{\tilde{\rho}(x)}\right) dx$$

In our setting, we define:

$$\tilde{\rho}(x) = \mu(T_\beta(x)) \cdot \left|\det\left(\frac{\partial T_\beta}{\partial x}(x)\right)\right|$$

so the divergence becomes:

$$KL[T_\beta] = \int \rho(x) \log\left(\frac{\rho(x)}{\mu(T_\beta(x)) \cdot \left|\det\left(\frac{\partial T_\beta}{\partial x}(x)\right)\right|}\right) dx$$

Using properties of logarithms, we rewrite:

$$KL[T_\beta] = \int \rho(x) \log \rho(x)\, dx - \int \rho(x) \log \mu(T_\beta(x))\, dx - \int \rho(x) \log\left|\det\left(\frac{\partial T_\beta}{\partial x}(x)\right)\right| dx$$

Now, suppose we have samples $\{x_i\}_{i=1}^n \sim \rho$. Then we approximate the remaining expectations using the Monte Carlo estimation:

$$\int \rho(x) f(x)\, dx \approx \frac{1}{n}\sum_{i=1}^n f(x_i)$$

Applying this, we get:

$$KL[T_\beta] \approx \text{const} - \frac{1}{n}\sum_{i=1}^n \log \mu(T_\beta(x_i)) - \frac{1}{n}\sum_{i=1}^n \log\left|\det\left(\frac{\partial T_\beta}{\partial x}(x_i)\right)\right|$$

Since the constant term $\int \rho(x) \log \rho(x)\, dx$ is independent of β, it can be omitted from the objective when optimizing. Therefore, we minimize:

$$KL[T_\beta] \approx \frac{1}{n}\sum_{i=1}^{n} \log\left(\frac{1}{\mu(T_\beta(x_i)) \cdot \left|\det\left(\frac{\partial T_\beta}{\partial x}(x_i)\right)\right|}\right)$$

Optimization

We minimize this approximate KL divergence with respect to β. Let $\bar{\beta}$ denote the optimal parameters:

$$\bar{\beta} = \arg\min_{\beta} KL[T_\beta]$$

Once we have the optimal map $G = T_{\bar{\beta}}$, we can estimate the original density ρ via:

$$\rho(x) = J^G(x) \cdot \mu(G(x)), \quad \text{where } J^G(x) = \left|\det\left(\frac{\partial G}{\partial x}(x)\right)\right|$$

This follows directly from the change-of-variable formula: we are pulling back the known density μ through the learned inverse map G to estimate ρ.

Mapping method

- So far we need an a-priori parametrization T_β rich enough to map ρ to a reference pdf (standard normal, say)
- Coming up with T_β is not an easy task $\rightarrow$ deep neural network
- The task becomes even harder if we want to impose specific characteristics on T_β like being optimal (theory of optimal transport) or triangular

$$\Downarrow$$

Normalizing Flows

Motivation for Normalizing Flows

Directly learning a single transformation $T_\beta : \mathbb{R}^d \to \mathbb{R}^d$ that maps a complex data distribution ρ to a simple reference distribution (such as a standard Gaussian) is a highly nontrivial task. Even with a deep neural

network, parameterizing such a function can be difficult due to issues with invertibility, stability, and efficient computation of the Jacobian determinant.

To address this, normalizing flows decompose the overall transformation into a sequence of simpler, invertible maps:

$$T = T_n \circ T_{n-1} \circ \cdots \circ T_1,$$

where each T_k is an easily parameterizable and tractable transformation (e.g., an affine coupling layer or autoregressive layer).
This layered construction offers several advantages:

- Each T_k can be made invertible by design, ensuring that the full map T is invertible.
- The Jacobian determinant of each T_k can be computed efficiently, allowing the use of the change-of-variables formula for exact density evaluation.
- The model becomes highly expressive while maintaining tractable training and sampling procedures.

Thus, rather than attempting to learn one large and complex transformation, normalizing flows build expressive models by composing multiple simple and structured transformations — making the learning process more stable, efficient, and interpretable.

Normalizing Flows (NF)

Main idea

Find $T = T_n \circ ... \circ T_1$ as a composition of elementary maps, easier to parametrize $T_k = T_{\beta^k}$

- Find T descending $KL[T(.,t)] = \int \rho \log(\rho/\tilde{\rho}_t)$:

$$\frac{dT(x,t)}{dt} = -\frac{\delta KL[\phi \circ T(.,t)]}{\delta \phi}\bigg|_{\phi=id} \tag{1}$$

where $T(x, t=0) = x$ and $\tilde{\rho}_t = J^{T(x,t)}\mu(T(x,t))$

In the end we have that $\tilde{\rho}_{t=0} = \mu$ and $\tilde{\rho}_\infty = \rho$

* Tabak, Esteban G., and Cristina V. Turner. "A family of nonparametric density estimation algorithms." Communications on Pure and Applied Mathematics 66.2 (2013): 145-164.
**Rezende, Danilo, and Shakir Mohamed. "Variational inference with normalizing flows." International conference on machine learning. PMLR, 2015.

Normalizing Flows

This slide explains how to transform one probability distribution into another using a series of simple, invertible functions. This is the key idea behind **Normalizing Flows (NF)**.

Main Idea

We want to build a complex transformation T by composing simpler ones:

$$T = T_n \circ T_{n-1} \circ \cdots \circ T_1$$

- It is difficult to directly design a function that maps a simple distribution (e.g., standard Gaussian) to a complex target distribution (e.g., real data).
- However, if we break the transformation into multiple steps, each step becomes easier to handle.
- Each map T_k can be parameterized by neural networks, denoted $T_k = T_{\beta_k}$, and trained using gradient-based optimization.

This is the fundamental strategy behind normalizing flows.

KL Divergence as the Objective

We want to learn a transformation T such that

$$T_{\#}\mu \approx \rho,$$

where:

- μ is the base (known) distribution, like a standard normal,
- ρ is the target distribution (e.g., empirical data),
- $T_{\#}\mu$ denotes the pushforward of μ by T, i.e., the distribution obtained by applying T to samples from μ.

To measure how close our current model $\tilde{\rho}_t$ is to the target ρ, we use the Kullback-Leibler divergence:

$$KL[\rho \| \tilde{\rho}_t] = \int \rho(x) \log \left(\frac{\rho(x)}{\tilde{\rho}_t(x)} \right) dx$$

Our goal is to find a transformation T that minimizes this KL divergence.

Key Equation: Functional Gradient Descent

To evolve T in a direction that reduces the KL divergence, we use:

$$\frac{dT(x,t)}{dt} = - \left. \frac{\delta KL[\phi \circ T(\cdot, t)]}{\delta \phi} \right|_{\phi = \text{id}} \tag{1}$$

Explanation:

- $T(x,t)$ is a time-dependent transformation.
- At $t = 0$, we initialize with $T(x, 0) = x$ (the identity map).
- The right-hand side is the functional derivative of KL divergence, which tells us how to adjust T to decrease KL most quickly.
- ϕ is a small perturbation applied to T, and we evaluate the derivative at $\phi = \text{id}$ (identity function).

This is analogous to gradient descent in parameter space, except we are descending in the space of functions. In normalizing flows, we don't typically work directly with this equation. Instead, it serves as a high-level description of how the transformation T should evolve over time to minimize KL divergence — guiding the flow in the infinite-dimensional space of functions.

Evolving the Density

At time t, the current density is given by:

$$\tilde{\rho}_t(x) = J^{T(x,t)}\mu(T(x,t)),$$

where $J^{T(x,t)}$ is the Jacobian determinant of the transformation T. This tells us how the probability density changes as we move through the transformation.

Initial and Final Conditions

We start with the base distribution:

$$\tilde{\rho}_{t=0} = \mu$$

and aim to reach the target:

$$\tilde{\rho}_{t\to\infty} = \rho$$

So as t increases, the transformation $T(x,t)$ warps the simple base distribution into the complex target distribution.

Example of Parameterizing the Flow

In the context of normalizing flows, we aim to construct a complex transformation

$$T = T_n \circ T_{n-1} \circ \cdots \circ T_1$$

that maps a simple base distribution μ, such as a standard Gaussian, to a complex target distribution ρ. Rather than defining each component map T_k by hand, we parameterize them using neural networks, enabling data-driven learning of the transformation.

Each map $T_k : \mathbb{R}^d \to \mathbb{R}^d$ is chosen to be invertible and differentiable. A widely used choice is the *affine coupling layer*, introduced in the RealNVP architecture. This map splits the input $x \in \mathbb{R}^d$ into two halves:

$$x = (x_a, x_b), \quad \text{where } x_a, x_b \in \mathbb{R}^{d/2}.$$

The transformation is then defined as:

$$\begin{aligned} y_a &= x_a \\ y_b &= x_b \odot \exp(s_\theta(x_a)) + t_\theta(x_a) \end{aligned}$$

Here:

- $s_\theta : \mathbb{R}^{d/2} \to \mathbb{R}^{d/2}$ is a neural network producing a log-scale vector,
- $t_\theta : \mathbb{R}^{d/2} \to \mathbb{R}^{d/2}$ is a neural network producing a shift vector,
- $\odot$ denotes elementwise multiplication,
- θ is the shared set of learnable parameters across both networks.

The exponential is applied elementwise to $s_\theta(x_a)$ to ensure positivity of the scaling factors, which guarantees invertibility of the transformation T_k. The overall flow step T_k is thus defined in terms of a pair of neural networks, both parameterized by θ_k. We write $T_k = T_{\theta_k}$, and collect all such parameters into $\beta = (\theta_1, \ldots, \theta_n)$.

Triangular Jacobian. The Jacobian matrix $J_{T_k}(x)$ of an affine coupling layer is lower triangular:

$$J_{T_k}(x) = \begin{bmatrix} I & 0 \\ * & \operatorname{diag}\left(\exp(s_\theta(x_a))\right) \end{bmatrix}$$

This triangular structure makes the determinant easy to compute:

$$|\det J_{T_k}(x)| = \prod_{j=1}^{d/2} \exp(s_\theta(x_a)_j) = \exp\left(\sum_{j=1}^{d/2} s_\theta(x_a)_j\right)$$

Training via Maximum Likelihood

Given a dataset $\{x^{(i)}\}_{i=1}^N$ sampled from the target distribution ρ, we aim to train the flow $T = T_n \circ \cdots \circ T_1$ such that the pushforward distribution $\tilde{\rho} := T_{\#}\mu$ closely approximates ρ.

This is done by maximizing the log-likelihood of the data under the model, or equivalently minimizing the negative log-likelihood:

$$\mathcal{L}(\beta) = -\sum_{i=1}^{N} \log \tilde{\rho}(x^{(i)}),$$

where β collects all parameters across all flow steps.

Change-of-Variables Formula

To compute $\tilde{\rho}(x)$, we apply the forward-mode change-of-variables formula where $z = T(x) \sim \mu$:

$$\tilde{\rho}(x) = \mu(T(x)) \cdot \left| \det \left(\frac{\partial T}{\partial x}(x) \right) \right|.$$

Substituting this into the loss gives:

$$\mathcal{L}(\beta) = -\sum_{i=1}^{N} \left[\log \mu(T(x^{(i)})) + \log \left| \det J_T(x^{(i)}) \right| \right].$$

Intuition Behind the Loss

The loss function consists of two interpretable terms:

- $\log \mu(T(x))$ encourages the model to learn transformations T that push observed data toward regions of high probability under the base distribution μ (typically a Gaussian).
- The Jacobian determinant $\left|\det J_{T(x)}\right|$ corrects for how much the transformation stretches or shrinks volume locally, ensuring proper density mass is preserved.

Together, these terms ensure that the model learns a transformation that both matches the shape and volume distortion of the target distribution.

Optimization

The loss $\mathcal{L}(\beta)$ is differentiable with respect to the neural network parameters β, and can therefore be minimized using stochastic gradient descent or adaptive optimizers such as Adam. Gradients are propagated through each transformation T_k, and in turn through the neural networks s_θ and t_θ that parameterize them. During training, the entire sequence of flow steps learns to approximate the target distribution ρ via optimization of the composite map T.

Introduction: Normalizing Flows (NF)

In practice:

- The flow

$$\frac{dT(x,t)}{dt} = -\frac{\delta KL[\phi \circ T(.,t)]}{\delta \phi}\bigg|_{\phi=id} \tag{2}$$

  is time discretized: $y^{n+1} = y^n + \phi(y^n, \theta^n)$ where ϕ is a perturbation of the identity map
- The resulting map $T(\cdot, t^n) = \phi^n \circ \phi^{n-1} \circ ... \circ \phi^1$ is the composition of elementary maps $\phi^k = \phi(\cdot, \theta^k)$

Why it is useful?

- Normalization, positivity constraints, curse of dimensionality and over-resolution make the parametrization of ρ a hard task.
- We don't need to parametrize ρ, we rather parametrize elementary maps that, through composition, form a rich family of one-to-one functions we use to recover ρ.

Baptista, Trigila, Wang PolyMathJr 2025: ML project 7 / 13

Normalizing Flows Continued

In normalizing flows, we often encounter the expression:

$$\frac{dT(x,t)}{dt} = -\frac{\delta}{\delta\phi}\mathrm{KL}[\phi \circ T(\cdot,t)]\bigg|_{\phi=\mathrm{id}}.$$

At first glance, this may seem circular: T is built from compositions of ϕ, so how can we apply ϕ to T again?

The key idea is this: at each time t, we **freeze** the transformation $T(\cdot, t)$ and treat it as fixed. Then, we apply a small perturbation ϕ on the left, forming $\phi \circ T$. We examine how the KL divergence changes as we vary ϕ, and compute the functional derivative with respect to ϕ, evaluated at the identity:

$$\frac{\delta}{\delta\phi}\mathrm{KL}[\phi \circ T]\bigg|_{\phi=\mathrm{id}}.$$

This tells us the direction in which to perturb T (via composition with ϕ) to most reduce the KL divergence.

Why the Negative Gradient Direction?

The gradient of a function always points in the direction of steepest ascent. So if our goal is to minimize a function—like KL divergence—we must move in the *opposite* direction: the negative gradient.

To build intuition, think of the function as a surface in 3D space, where the height $z = f(x, y)$ represents the value of the function. At any point on this surface, the gradient $\nabla f(x, y)$ takes in your current x and y coordinates and gives you a direction in the xy-plane. That direction tells you how to walk in x and y to increase z as quickly as possible. In other words, the gradient is like a compass that points straight uphill.

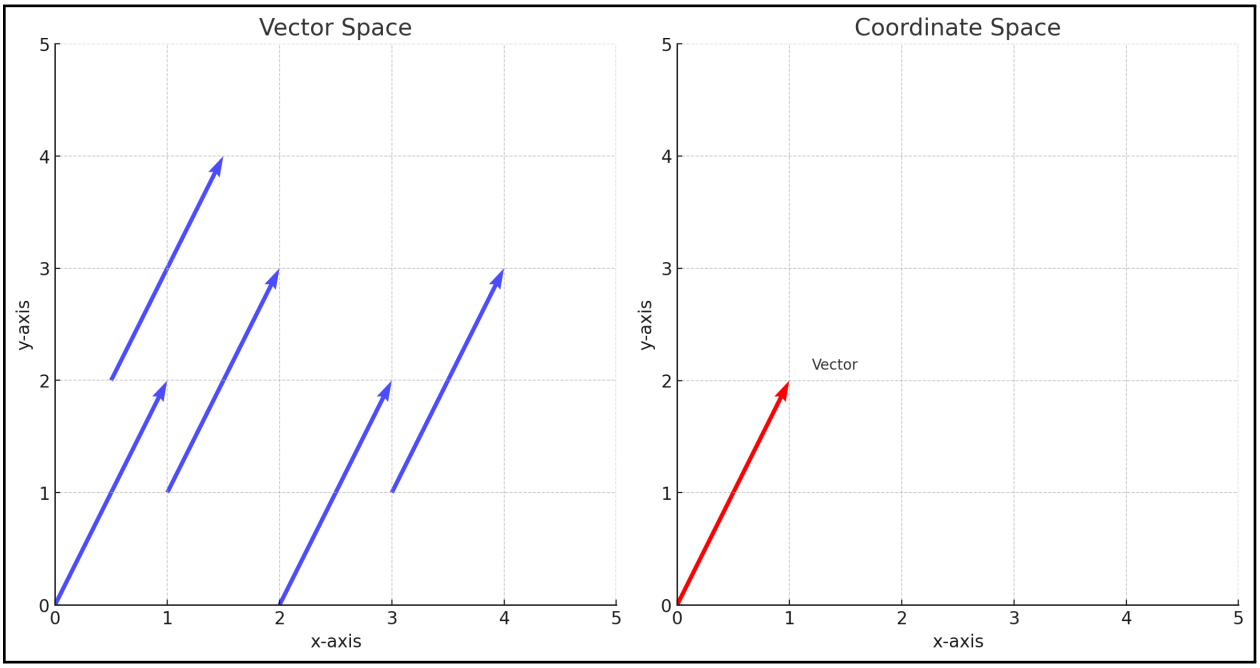
But if you want to *decrease* z, you walk in the opposite direction—downhill. That's why in gradient descent, we follow $-\nabla f$: it's the fastest way to reduce the function value.

In our case, we're minimizing not over scalars or vectors, but over functions. The KL divergence is a functional, and the derivative

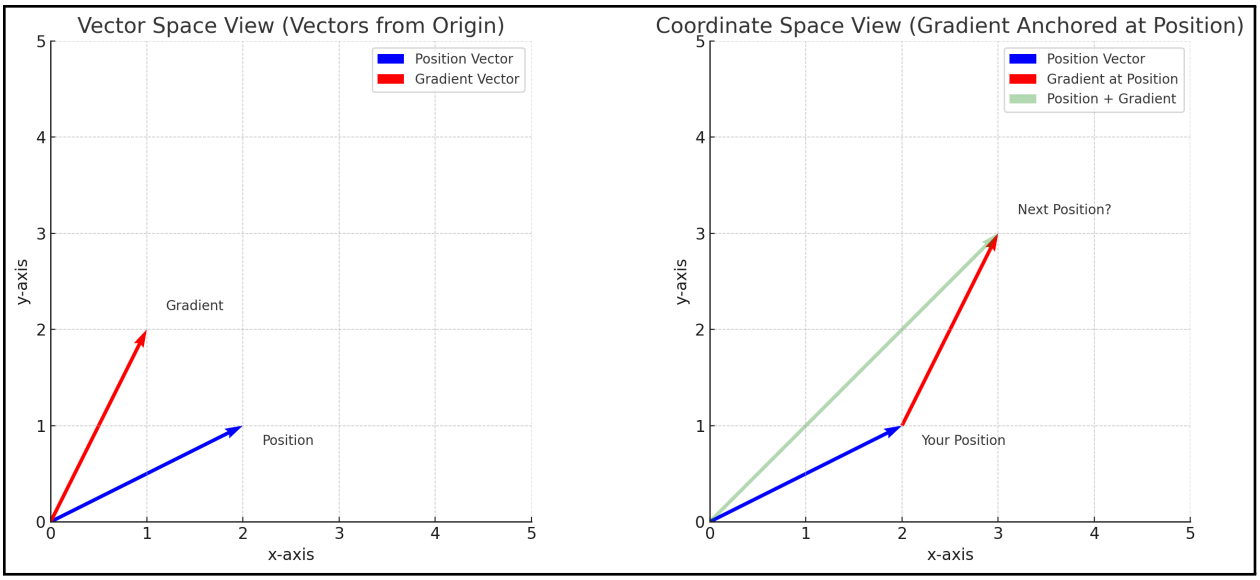
$$\frac{\delta}{\delta \phi} \mathrm{KL}[\phi \circ T]\Big|_{\phi=\mathrm{id}}$$

acts like a gradient in function space. It tells us: if we compose T with a small perturbation ϕ, how does KL change? This is our generalization of the compass. And just like before, we move in the *negative* direction to descend KL as efficiently as possible.

Note on Canonical Isomorphism and Vector Addition



Under the canonical isomorphism between a vector space and its coordinate representation, all vectors are drawn with their tails at the origin. This isomorphism allows us to represent vectors in a vector space as coordinate tuples in $\mathbb{R}^n$, anchored at the origin.



When we want to add vectors in coordinate space, we place them tip-to-tail. For example, to apply a gradient vector to a position vector, we interpret both as anchored at the origin, then translate the gradient vector to start at the tip of the position vector. The result of the addition is the vector from the origin to the tip of the second (translated) vector.

Optimal transport framework to map PDFs

Problem

How to move a pile of sand to fill a pit of the same volume minimizing a given cost?

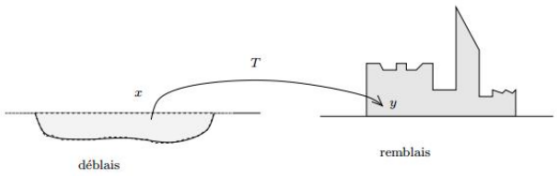


Fig. 3.1. Monge's problem of déblais and remblais

Math formulation

Given two probability densities, $\rho(\mathbf{z})$ and $\mu(\mathbf{z})$, $\mathbf{z} \in \mathbb{R}^d$, find a 1-to-1 map $T : \mathbb{R}^d \to \mathbb{R}^d$ such that $T_{\#\rho} = \mu$ and that minimizes the functional

$$M[T] = \int_{\mathbb{R}^d} c(\mathbf{z}, T(\mathbf{z}))\rho(\mathbf{z})d\mathbf{z}$$

The minimizer is called an *optimal transport map.*

Baptista, Trigila, Wang PolyMathJr 2025: ML project 8 / 13

The Optimal Transport Problem

The classical *Optimal Transport* (OT) problem asks:

> *Given two probability distributions, how can we transform one into the other in the most cost-efficient way?*

This is often described using the metaphor: "How do you move a pile of sand to fill a hole of the same volume, while minimizing the total effort required?"

Mathematical Setup

Let $\rho(z)$ and $\mu(z)$ be two probability density functions on $\mathbb{R}^d$. The goal is to find a measurable, invertible map $T : \mathbb{R}^d \to \mathbb{R}^d$ such that the *pushforward* of ρ by T equals μ:

$$T_{\#}\rho = \mu.$$

That is, when we apply T to samples drawn from ρ, the resulting distribution becomes μ.

Cost Functional

Each movement of a point $z \in \mathbb{R}^d$ to a new location $T(z)$ incurs a *cost*, denoted by $c(z, T(z))$, where $c : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ is a user-defined cost function (such as squared Euclidean distance).

The total transport cost across the entire distribution is defined by the functional:

$$M[T] = \int_{\mathbb{R}^d} c(z, T(z))\, \rho(z)\, dz.$$

This integral has the following interpretation:

- $\rho(z)$ is a probability density — it represents the instantaneous probability per unit volume at the point z.
- For each infinitesimal region around $z \in \mathbb{R}^d$, the product $\rho(z)\, dz$ gives the amount of "mass" located there (the probability mass in that tiny volume).
- The per unit cost to move that mass to its destination $T(z)$ is given by $c(z, T(z))$.
- Multiplying them gives the local transport cost: $c(z, T(z)) \cdot \rho(z)\, dz$.
- Integrating this quantity over all z accumulates the total cost of transporting the full distribution.

Such a map T^* is called an *optimal transport map.*

Summary Table: Definite vs. Indefinite Integrals

Type	Notation	Output	Meaning
Definite Integral	$\int_a^b f(x)\, dx$	Number	Net change a to b
Indefinite Integral	$\int f(x)\, dx$	Function $+ C$	Family of antiderivatives

Example: Interpreting Probability Densities in 2D

Let's say you have a 2D random variable $z = (z_1, z_2) \in \mathbb{R}^2$, and a probability density function $\rho(z)$.

- Then $\rho(z)$ has units of *probability per unit area.*
- If $\rho(z) = 0.25$ at some point, this means that in a small square of area 0.01 around that point, there is approximately $0.25 \times 0.01 = 0.0025$ units of probability mass.
- When we write:

  $$\int_{\mathbb{R}^2} \rho(z)\, dz = 1,$$

  we are saying that the *total probability mass* over the entire 2D space is 1.

How This Relates to Integration Types

In this example:

- The expression $\int_{\mathbb{R}^2} \rho(z)\, dz$ is a **definite integral**. It outputs a number (in this case, 1), representing the total probability mass over all of $\mathbb{R}^2$.
- If you were instead given the function $\rho(z)$, and wanted to find its antiderivative (which isn't typical for probability densities), that would be an **indefinite integral**. It would return a function $F(z) + C$, whose gradient (or divergence) gives you back $\rho(z)$.

Thus:

- **Definite integrals** are used to calculate *quantities*, like probability mass, area, or net change.
- **Indefinite integrals** are used to *recover functions* from their derivatives, up to an arbitrary constant.

Quadratic cost

Consider quadratic cost:

$$M[T] = \int_{\mathbb{R}^d} |\mathbf{z} - T(\mathbf{z})|^2 \rho(\mathbf{z}) d\mathbf{z}$$

If ρ and μ are smooth enough and have finite second moment, the optimal (Brenier) map exists, is unique and is given by the gradient of a convex function $\phi : \mathbb{R}^d \to \mathbb{R}$ satisfying the Monge-Ampere (MA) equation:

$$\rho(\mathbf{z}) = \mu(\nabla\phi(\mathbf{z})) \det(D^2\phi(\mathbf{z}))$$

One way of finding the optimal map from ρ to μ is to solve the MA equation enforcing convexity of the solution

Baptista, Trigila, Wang PolyMathJr 2025: ML project 9 / 13

We consider the problem of optimal transport between two probability densities ρ and μ defined over $\mathbb{R}^d$. A key ingredient in formulating the transport problem is the **cost functional**, which quantifies how "expensive" it is to move mass from one point to another.

Quadratic Cost

Definition (Quadratic Cost). The *quadratic cost* refers to the squared Euclidean distance between source and target locations in the transport map. That is, if $T : \mathbb{R}^d \to \mathbb{R}^d$ is a candidate transport map, the cost of transporting ρ to μ using T is defined as:

$$M[T] = \int_{\mathbb{R}^d} \|\mathbf{z} - T(\mathbf{z})\|^2 \, \rho(\mathbf{z}) \, d\mathbf{z}$$

Here:

- $\rho(\mathbf{z})$ is the density of the source measure,
- $T(\mathbf{z})$ is the image of $\mathbf{z}$ under the transport map,
- $\|\mathbf{z} - T(\mathbf{z})\|^2$ is the *pointwise cost*.

The goal of optimal transport is to find the map T that *pushes forward* ρ to μ (i.e., $T_{\#}\rho = \mu$) while minimizing $M[T]$. Remember, when we apply T to samples drawn from ρ, the resulting distribution becomes μ.

Brenier's Theorem and the Convex Potential

To understand the structure of optimal transport maps under quadratic cost, we begin by defining two key ideas: the *second moment* of a distribution, and the concept of a *convex potential function*.

Second Moment

Let $X \in \mathbb{R}^d$ be a random vector with probability density function ρ. The **second moment** of ρ is defined as:

$$\mathbb{E}[\|X\|^2] = \int_{\mathbb{R}^d} \|\mathbf{z}\|^2 \, \rho(\mathbf{z}) \, d\mathbf{z}$$

Here:

- $\mathbf{z} = (z_1, \ldots, z_d) \in \mathbb{R}^d$ is a point in space,
- $\|\mathbf{z}\|^2 = z_1^2 + z_2^2 + \cdots + z_d^2$ is the square of the Euclidean norm (i.e., squared distance from the origin),
- $\rho(\mathbf{z})$ is the value of the probability density at point $\mathbf{z}$.

This integral gives the expected squared distance of a sample drawn from ρ relative to the origin. A distribution is said to have a *finite second moment* if this integral is finite.

Convex Potential Function

A function $\phi : \mathbb{R}^d \to \mathbb{R}$ is called **convex** if it satisfies:

$$\phi(\lambda \mathbf{z}_1 + (1-\lambda)\mathbf{z}_2) \leq \lambda \phi(\mathbf{z}_1) + (1-\lambda)\phi(\mathbf{z}_2) \quad \text{for all } \lambda \in [0,1]$$

Intuitively, this means that *the graph of the function lies below the straight line connecting any two points on it.* There are no dips or valleys between any two points.

A **convex potential function** is just a convex function ϕ whose gradient $\nabla\phi$ defines a transport map.

Brenier's Theorem

Brenier's Theorem states the following:
If ρ and μ are two absolutely continuous probability densities on $\mathbb{R}^d$ with finite second moments, then there exists a unique optimal transport map T^* pushing ρ forward to μ, and it is given by:

$$T^*(\mathbf{z}) = \nabla\phi(\mathbf{z})$$

where:

- $\phi : \mathbb{R}^d \to \mathbb{R}$ is a convex potential function,
- $\nabla\phi(\mathbf{z})$ is the gradient vector, describing where to send mass originally located at $\mathbf{z}$.

So under quadratic cost, the optimal way to transport mass is to follow the gradient of a convex potential.

Note: Interpreting the Gradient in Optimal Transport

In standard calculus:

- $\nabla\phi(\mathbf{z})$ gives the direction and rate of steepest increase of ϕ at the point $\mathbf{z}$.

But in **optimal transport theory** — especially under **Brenier's Theorem** — we re-interpret the gradient $\nabla\phi$ as a function that maps each point $\mathbf{z}$ to a new point in space.

In this context, $\nabla\phi(\mathbf{z}) \in \mathbb{R}^d$ is not just a direction — it's interpreted as the **destination of mass located at $\mathbf{z}$**.

Absolutely Continuous Measures

A probability measure ρ on $\mathbb{R}^d$ is said to be **absolutely continuous with respect to Lebesgue measure** if it has a **density function** — that is, there exists a function $\rho(\mathbf{z})$ such that:

$$\mathbb{P}(A) = \int_A \rho(\mathbf{z})\, d\mathbf{z} \quad \text{for all measurable sets } A$$

In other words:

You can write the measure as an integral with respect to the standard volume in $\mathbb{R}^d$.
This means ρ "doesn't concentrate" on points, lines, or lower-dimensional shapes — it's spread out in space.

Note: Why Lower-Dimensional Sets Have Zero Lebesgue Measure

Because in $\mathbb{R}^d$, the Lebesgue measure is defined in terms of **volume** (i.e., d-dimensional volume), lower-dimensional subsets have zero measure.

- In $\mathbb{R}^1$: points have zero measure, intervals have positive measure.
- In $\mathbb{R}^2$: lines (1D objects) have zero area.
- In $\mathbb{R}^3$: surfaces (2D objects) have zero volume.
- More generally: any set of dimension $< d$ has zero d-dimensional Lebesgue measure.

Monge–Ampère Equation

To find the optimal convex function ϕ, one must solve a partial differential equation known as the **Monge–Ampère (MA) equation**:

$$\rho(\mathbf{z}) = \mu(\nabla\phi(\mathbf{z})) \cdot \det(D^2\phi(\mathbf{z}))$$

This equation expresses the condition that the gradient map $\nabla\phi$ preserves mass when transporting ρ to μ.
Term-by-term explanation:

- $\rho(\mathbf{z})$: Source density at $\mathbf{z}$,
- $\nabla\phi(\mathbf{z})$: Target point for mass at $\mathbf{z}$,
- $\mu(\nabla\phi(\mathbf{z}))$: Target density at the image point,
- $D^2\phi(\mathbf{z})$: Hessian matrix of second derivatives of ϕ,
- $\det(D^2\phi(\mathbf{z}))$: Jacobian determinant — volume distortion under $\nabla\phi$.

From Monge–Ampère to Optimal Transport Map

To get the **optimal transport map** T^* under **quadratic cost**, Brenier's Theorem tells us:

$$T^*(z) = \nabla\phi(z),$$

for some **convex potential function** ϕ.
But not just any convex function ϕ will do — it must be the one that **solves the Monge–Ampère equation**:

$$\rho(z) = \mu(\nabla\phi(z)) \cdot \det(D^2\phi(z)).$$

One idea on OT

We need to impose the convexity of the potential → we want the map to be the gradient of a convex function. If we build the flow

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^{k} \nabla_x F(z^i)$$

with $z^0 = x$ the starting position, $\beta \in R$, and F convex, then the convexity of the potential depends on the sign of β.

- When β^i is positive there is no problem (sum of convex functions is still convex)
- If β is negative we need to pay attention
- We only have the value $\sum_{i=1}^{k} \nabla_x F(z^i)$ at the sample points z^i
- A condition that $\sum_{i=1}^{k} \nabla_x F(z^i)$ should satisfy is that, at least, should interpolate a convex function

Constructing Convex Transport Maps via Gradient Flow

In the context of optimal transport under quadratic cost, Brenier's Theorem guarantees that the optimal transport map T^* is the gradient of a convex potential function:

$$T^*(x) = \nabla \phi(x),$$

for some convex function $\phi : \mathbb{R}^d \to \mathbb{R}$. The goal of this construction is to build such a map iteratively using gradient flows, ensuring that the resulting map maintains the necessary convexity structure.

Discrete Flow Construction

We define a sequence of points $\{z^k\} \subset \mathbb{R}^d$ via the recurrence:

$$z^{k+1} = z^k + \beta^k \nabla F(z^k),$$

where:

- $z^0 = x \in \mathbb{R}^d$ is the initial point,
- $\beta^k \in \mathbb{R}$ is a step-size coefficient at iteration k,
- $F : \mathbb{R}^d \to \mathbb{R}$ is a known convex function.

Expanding this recursion yields:

$$z^{k+1}(x) = x + \sum_{i=1}^{k} \beta^i \nabla F(z^i),$$

which defines a transformation of the form:

$$T(x) := z^{k+1}(x).$$

Convexity Considerations

The transport map $T(x)$ is intended to approximate the gradient of a convex potential function. Our construction achieves this under the following conditions:

- **Positive weights:** Convex functions are closed under positive linear combinations. Since the gradient of a sum of convex functions (with positive weights) is equivalent to the sum of their gradients, the resulting transformation remains the gradient of a convex function.
- **Negative weights:** If any $\beta^i < 0$, convexity may be lost. In this case, the sum of gradients may no longer correspond to the gradient of a convex function. Careful control of step sizes is therefore necessary to preserve the desired structure.
- **Sampled gradients:** In practice, the gradients $\nabla F(z^i)$ are computed only at a finite collection of sample points z^i. The map $T(x)$ is therefore constructed by stitching together directional updates at these discrete locations. However, Brenier's theorem requires the overall transformation to behave like the gradient of a single globally defined convex function. As such, the sum

  $$\sum_{i=1}^{k} \beta^i \nabla F(z^i)$$

  must approximate—at least locally—the gradient of a convex potential. This imposes structural constraints on the sampling strategy, the choice of weights β^i, and the properties of the function F itself.

Interpretation and Goal

This iterative construction defines a flow of the point x under successive applications of directional updates determined by ∇F. The final position $z^{k+1}(x)$ defines a map that can be interpreted as a transport map:

$$T(x) = z^{k+1}(x),$$

provided it satisfies the convexity condition.

Research opportunity

Main goal of this project

Given samples $(\mathbf{z}^i)$ drawn from the unknown $\rho(\mathbf{z})$ we want to build a new generative model that map samples from $\rho_0(\mathbf{z})$ to $\rho(\mathbf{z})$.

Avenues for numerical exploration:

- Implement the data-driven OT flow algorithm with adaptive bandwidth and kernel
- Evaluate convergence when using an improved back-and-forth procedure

Avenues for theoretical exploration:

- Show the equivalence between the data-driven and maximum mean discrepancy flow
- Mathematical understanding of back-and-forth procedure

Delve into applications:

- Evaluation on 2D benchmark problems (banana, checkerboard)
- Image generation and solving Bayesian inference problems

Thank you!
Questions?

References

- Marzouk et. al, An introduction to sampling via measure transport, Handbook of UQ, 2016
- Papamakarios et. al, Normalizing Flows for Probabilistic Modeling and Inference, JMLR, 2021
- Santambrogio, Optimal Transport for Applied Mathematicians, Springer, 2015
- Tabak E.G., Trigila G. Data-driven optimal transport, Communications on Pure and Applied Math, 2016
- Galashov, de Bortoli, Gretton, Deep MMD Gradient Flow without adversarial training, 2024